

EPSRC

Matty Boles

July 2026

1 Introduction

Data-driven prediction of chaotic systems using machine learning methods is an ever-changing research field, important for improving accuracy of forecasting in areas such as weather prediction. Neural networks offer significant computational advantages over classical methods for chaotic systems, but their utility depends on preserving the chaotic properties of the true system. The mechanisms determining whether, and how, a network preserves chaos, remains systematically unanalysed. This is the objective of this project, using a variety of stability analysis techniques, introduced shortly, to compare a variety of neural networks on the Lorenz-63 equations.

1.1 Chaotic Dynamical Systems

Chaotic systems are dynamical systems in which infinitesimal perturbations of initial conditions cause trajectories to separate exponentially. Although they are deterministic, they exhibit aperiodic behaviour. Due to tiny errors compounding, accurate long-term prediction of chaotic systems, past a finite time horizon, is impossible. Even floating point errors can cause the prediction to exponentially separate from the true solution. For this reason, we focus the project on learning the long-term statistical and dynamical properties of a system, which are consistent throughout initial conditions.

1.2 Analysis techniques

We have 3 general measures of a successful model:

1.2.1 Lyapunov Spectrum

The Lyapunov spectrum of a system describes the rate of exponential separation in orthogonal directions. The first, largest value, the Lyapunov Exponent, λ_1 , dictates the overall separation of the system:

$$||x'(t)|| \sim e^{\lambda_1 t} ||x'(0)||,$$

where $x'(t)$ is the perturbation at time t . Hence, by rearranging, we find λ_1 as follows:

$$\lambda_1 = \lim_{t \rightarrow \infty} \frac{1}{t} \ln \frac{||x'(t)||}{||x'(0)||}$$

To make this calculation easier, we define the tangent propagator, $M(t, t_0)$, which describes the evolution of the perturbation from time t_0 to t :

$$x'(t) = M(t, t_0)x'(t_0)$$

As the choice of initial time is arbitrary, we set $t_0 = 0$. Then, by substituting and rearranging, we obtain:

$$\lambda_1 = \lim_{t \rightarrow \infty} \frac{1}{t} \ln \frac{||M(t, 0)x'(0)||}{||x'(0)||}$$

By assuming perturbations are small, we can see they show linearised dynamics:

$$\dot{x}'(t) = J(t)x'(t)$$

Here, $J(t) = F'(x)|_{x=x(t)}$. Then, we see M satisfies:

$$\dot{M}(t, 0) = J(t)M(t, 0), \quad M(0, 0) = I,$$

which has the formal solution,

$$M(t, 0) = \mathcal{T} \exp \left(\int_0^t J(s) ds \right),$$

where, $\mathcal{T}$ is the time-ordering operator. A more in-depth proof is provided in the Appendix A.

The Lyapunov Exponent defines the overall chaos of a system:

- $\lambda_1 < 0$ is stable.
- $\lambda_1 = 0$ is periodic.
- $\lambda_1 > 0$ is chaotic.

More generally, the Lyapunov spectrum describes this in orthogonal directions, and is found by:

$$\lambda_i = \lim_{t \rightarrow \infty} \frac{1}{t} \ln \sigma_i(M(t, 0)),$$

where σ_i is the i th singular value. A perfect model would have a Lyapunov spectrum close to the target spectrum.

1.2.2 Singular Value Decomposition

The singular value decomposition (SVD) of the Jacobian of the system describes local geometry by identifying orthogonal directions of instantaneous stretching and contraction. Similarly to before, however in linear space rather than exponential, a singular value greater than 1 describes stretch, and less than 1 describes contraction. SVD represents a matrix, A , as :

$$A = U \Sigma V^T,$$

where:

- V contains the right singular vectors, which form an basis of the input space.
- Σ contains the singular values, σ_i , which are stretching factors.
- U contains the left singular vectors, which form a basis of the output space.

We focus our attention on σ_i , to ensure a well-trained model successfully recreates the local geometry throughout the system. Indeed, a model can have perfect a perfect Lyapunov spectrum, as it is a long-term average, and fail to capture the local geometry at every point in the space.

1.2.3 Attractor statistics and shape

The final metric of a successful model is the statistical properties of the model. We look at Probability Density Functions (PDFs) and histograms of state variables and features of the system, as well as in particular, the visitation rate of rare regions of phase space. A common failure mode of neural networks is that they systematically under-predict rare outcomes. This is a prominent problem in weather forecasting, where neural networks are effective for 'typical' atmospheric states, but fail to predict extreme events such as heatwaves and severe storms. These events have disproportionate impacts, and so accuracy and distribution are key.

1.3 Lorenz-63

The Lorenz-63 equations portray a simplified system for convection cycles in the atmosphere. They were developed by Edward Lorenz in his 1963 paper, and have served as a test bed for non-linear dynamics since, due to the simplicity of the equations, and incredibly detailed, structured and sensitive resulting geometry.

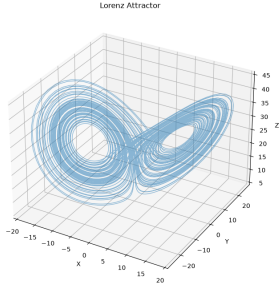
$$\begin{aligned}\dot{x} &= \sigma(y - x) \\ \dot{y} &= x(\rho - z) - y \\ \dot{z} &= xy - \beta z\end{aligned}$$

Here, x is proportional to the intensity of the convection motion, y is proportional to the temperature difference between ascending and descending currents, and z is proportional to the distortion of the vertical temperature profile from linearity. σ is the Prandtl number, ρ is the Rayleigh number and β is another, physical constant. We use the typical values, which ensure chaos, of $\sigma = 10, \beta = \frac{8}{3}, \rho = 28$.

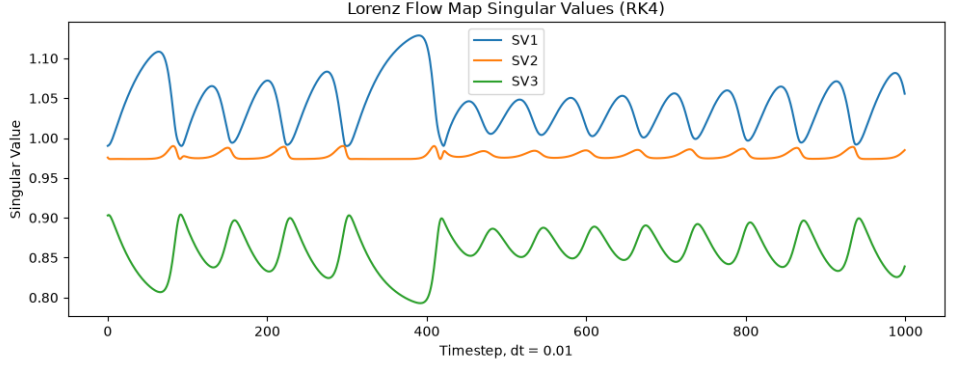
These parameters of Lorenz give rise to the classical butterfly attractor. This is a 2-lobed shape, in which attracted trajectories circle an unstable centre, until sufficient distance is reached, at which time the trajectory leaves the cycle of one centre and swaps to the other centre. This provides a system where it is difficult to predict the number of loops of a centre, before swapping. This gives rise to the chaos. The system has 1 saddle point at $(0,0,0)$, as well as the 2 unstable centres at $(\pm\sqrt{\beta(\rho-1)}, \pm\sqrt{\beta(\rho-1)}, \rho-1)$, which, with our parameters, are $(\pm6\sqrt{2}, \pm6\sqrt{2}, 27)$.

The Lyapunov spectrum of the true Lorenz system is roughly:

- $\lambda_1 = 0.906$
- $\lambda_2 = 0$



(a) The Lorenz attractor, with initial point (2,4,9), iterated for 10000 time steps of length $dt = 0.01$, using the RK4 method.



(b) The flow map singular values of the true Lorenz system, over 1000 time steps, with $dt = 0.01$, using the RK4 method.

Figure 1: Lorenz Attractor and SVD of Jacobian

- $\lambda_3 = -14.57$

The singular value decomposition of the Lorenz Jacobian shows that the singular value describing stretching, σ_1 , oscillates between around 1 and 1.13, for each loop of a centre. The amplitude of the oscillations is proportional to the width of the loop, i.e. the amplitude of σ_1 grows with the radius of a loop, and after lobe switching, the amplitude drops back down. Similarly, σ_3 oscillates with amplitude proportional to the width of the current loop, within the range 0.77 and 0.91.

1.3.1 Model Overview

One idea of this project is to focus on the minimum viable network, and many decisions were made with this in mind.

- This project focuses on simple feed forward networks, as we try to justify why more complicated models are preferred or required.
- We focus on only 1 layer of hidden neurons.
- A small number of hidden neurons. We know an arbitrarily large system can approximate the Lorenz system with ease, and as such we are uninterested.
- We initially only use 1-step Mean Squared Error (MSE) as a loss scheme. Indeed we have said that the focus is not on differences between the trajectories, and later in the paper we turn our focus to biasing the loss to geometric features of the true system, but it is an interesting finding what the model can learn based purely on error of trajectories, as the simplest solution.

All models were trained with pairs of inputs and output coordinates in (x,y,z). Care was taken to ensure all points are on the attractor by integrating over a 'transient period' which was then disregarded. Input and output pairs are found using the 4th order Runge-Kutta (RK4) time stepping scheme, to discretise the solution.

2 Training Hyperparameters

It may be many data scientist first thought to use an AdamW optimiser and call it a day. In fact this comes with 2 issues. The first is weight decay. Weight decay is regularization technique used by AdamW as standard to reward simpler solutions by punishing weights of large magnitude. For parameter θ ,

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t & v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, & \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \\ \theta_t &= \theta_{t-1} - \gamma \frac{\hat{m}_t}{(\sqrt{\hat{v}_t} + \epsilon)} - \gamma \lambda \theta_{t-1} \end{aligned}$$

Here, t is the optimisation step, g is the gradient of the parameter with respect to the loss function, γ is the learning rate, and λ is the weight decay.

As the Lorenz system is deterministic and contains no noise, the model can (theoretically) converge to an exact minima. However, the weight decay continually pulls the parameters back towards 0, regardless of value, including if it were already

optimal. This regularization is helpful in large, noisy datasets, where it helps to prevent overfitting, it harms dynamical systems which require accurate representation of the governing equations. In these cases, even tiny perturbations in weights can have large changes on the dynamics of the system.

In particular, in the Lorenz-63 system, there are areas of large contraction and expansion, caused by non-linearities. These require relatively large weights to portray, and so a model with smaller weights than required lacks the peaks and troughs in the singular values of the Jacobian, which results in a more 'typical trajectory', in which loops are of more consistent radius, and rare areas of the map are not visited, such as narrow loops to the unstable points.

The second problem is the way AdamW (Or Adam) calculates the weight updates. We assume weight decay is 0. The main issue is that the weight updates of Adam only indirectly interact with other parameters, rather than explicitly having cross-parameter coupling. The advantages of parameter-wise optimisation are less appropriate when the number of parameters is small and they are highly coupled. However, in such small networks, all neurons are directly influential on each other. The Adam optimiser has no explicit representation of cross-parameter curvature, as 2nd moment influences are only parameter-wise. Each weight update is scaled with only itself as context. In a dynamical system such as this, some weight updates are much larger than others, which we call being 'ill-conditioned'. Adam doesn't have this context, and so determines the scale of each update using primarily information about that parameter, rather than as a collection of closely coupled parameters. E.g. a big step in a direction may cause another weight to miss it's minima.

We think of the loss landscape as a valley. AdamW finds the valley well. However, once in the valley, we have a direction of high curvature and a direction of low curvature (ill-conditioned). Adam has no context of the local terrain, and performs parameter-wise rescaling, partially removing differences in gradient magnitude which are informative. This produces ineffective trajectories across the narrow directions of the valley, rather than following the direction of overall descent, which is a common problem for optimisers which do not use local curvature information.

By calculating the Hessian of loss with respect to parameters when training slows, we can show evidence of this failure mode. We ran a 4-neuron model for 2000 epochs with the AdamW optimiser, no weight decay. The resulting 31 x 31 Hessian has a minimum eigenvalue of -0.00217 , minimum absolute eigenvalue of $4.05e^{-6}$, and a maximum eigenvalue of 180.52. This effectively shows the problem is ill-conditioned, and AdamW would struggle to converge. Furthermore, by finding the Frobenius norm of the non-diagonal terms over the norm of the full matrix, we find a coupling ratio of 0.66, which implies weights are highly coupled.

We suggest LBFGS loss instead. Indeed, not only models converge much quicker to a stable MSE, and with much fewer samples, but the MSE converges to a much lower value. This is due to the use of an estimate of the inverse Hessian in the update scheme, providing knowledge of the local curvature, and coupling of parameters. L-BFGS uses the BFGS approximation for the Hessian. For optimisation step k , define:

$$\begin{aligned} s_k &= x_{k+1} - x_k & y_k &= g_{k+1} - g_k \\ \rho_k &= \frac{1}{y_k^T s_k} \\ H_{k+1} &= (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T \end{aligned}$$

This allows gradient descent in the more effective direction: where Adam rescales coordinates independently, L-BFGS transforms the joint gradient according to the local curvature. In the earlier valley analogy, where Adam is zig-zagging between the sharp cliffs of the valley, LBFGS follows the valley down to the minimum.

3 Activation Functions

An activation is a function applied to each neuron before output, to introduce non-linearity. Choice of activation function can dramatically influence the ability of a model to learn a solution. It is even more significant in dynamical systems because the activation function directly contributes to the Jacobian, which we have seen describes the dynamics. As in this paper, we assume a 1-layer MLP, with weights W_1, W_2 , biases b_1, b_2 and activation function ϕ :

$$\begin{aligned} z &= W_1 x + b_1 \\ a &= \phi(z) \\ \hat{y} &= W_2 a + b_2 \end{aligned}$$

The Jacobian of the model, J , is:

$$\begin{aligned} J(x) &= \frac{d}{dx} (W_2 a + b_2) = W_2 \frac{d}{dx} (\phi(z)) \\ &= W_2 \frac{d(\phi(z))}{dz} \frac{dz}{dx} \\ &= \sum_i \frac{d\phi}{dz_i} W_{2, :, i} W_{1, i, :} \end{aligned}$$

Here, we see $J(x)$ is the sum of many rank-1 matrices, one contribution for each neuron.

3.1 ReLU

We first investigate ReLU, a common activation function, and examine it's unsuitability for capturing the dynamics of the Lorenz system:

$$ReLU(z) = \begin{cases} 0, & \text{if } z < 0, \\ z, & \text{if } z \geq 0 \end{cases}$$

We see that ReLU is piecewise linear, in Figure 2. We will show that ReLU fails to capture the dynamics of the Lorenz equations.

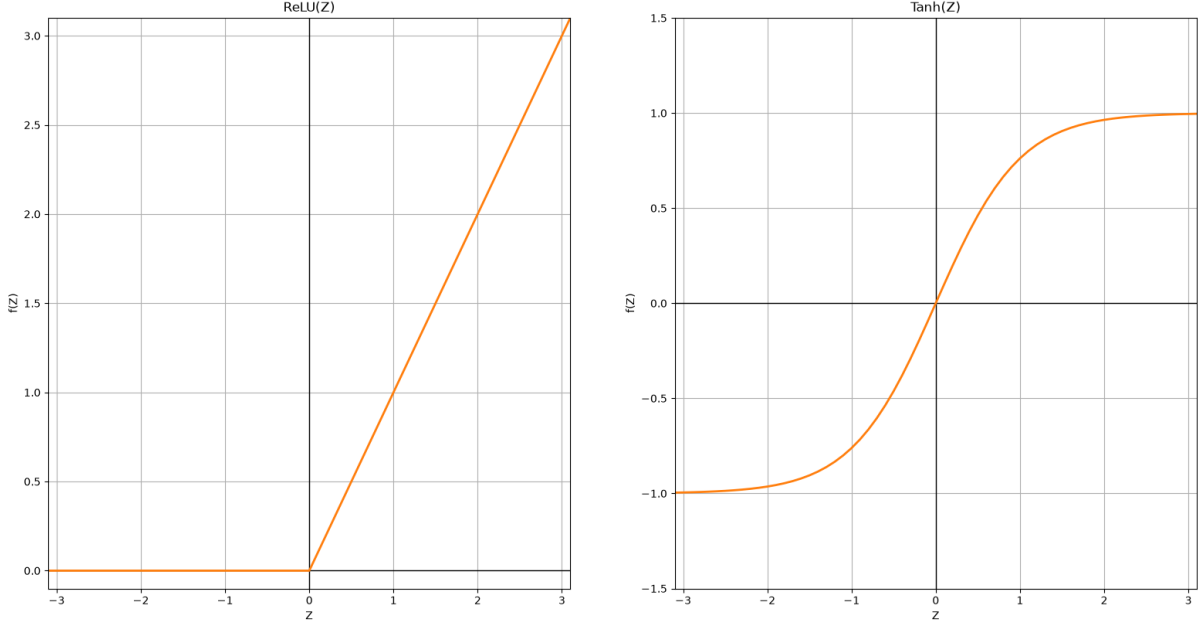


Figure 2: (Left) $ReLU(z)$ activation function (Right) $Tanh(z)$ activation function

A model was trained of width 32, for multiple seeds, and analyse the best performance. The Lyapunov spectrum initially looks passable, with:

- $\lambda_1 = 0.288$
- $\lambda_2 = -0.091$
- $\lambda_3 = -14.15$

This shows expansion in one direction, and contraction in another, just like the true Lorenz.

However, the inspection of the SVD shows discontinuous behaviour incompatible with the true Lorenz geometry, shown in Figure 3. The sinusoidal shape still vaguely appears, however, unlike the true system, amplitude doesn't appear to correlate with loop radius. There are very sharp drops in $\hat{\sigma}_3$, the contracting direction, to account for the fold areas.

This is to be expected, as the derivative of ReLU is:

$$\frac{d}{dz}(ReLU(z)) = \begin{cases} 0, & \text{if } z < 0, \\ 1, & \text{if } z > 0 \end{cases}$$

(Note: Although technically undefined, we practically take $\frac{d}{dz}(ReLU(z))|_{z=0} = 0$, for computational convenience)

The derivative of neurons changes abruptly when a neuron passes from negative to positive. Hence, each neuron's rank-1 contribution is either 0, when $z_i \leq 0$, or $W_{2,:i}W_{1,i,:}$, when $z_i > 0$. This switches discontinuously over the trajectory. Hence the Jacobian also varies discontinuously, changing only when a neuron changes sign. This means the singular values and vectors only ever change in discrete jumps. As a result, a ReLU network cannot represent the continuously varying rotations, contractions and expansions required by the Lorenz system.

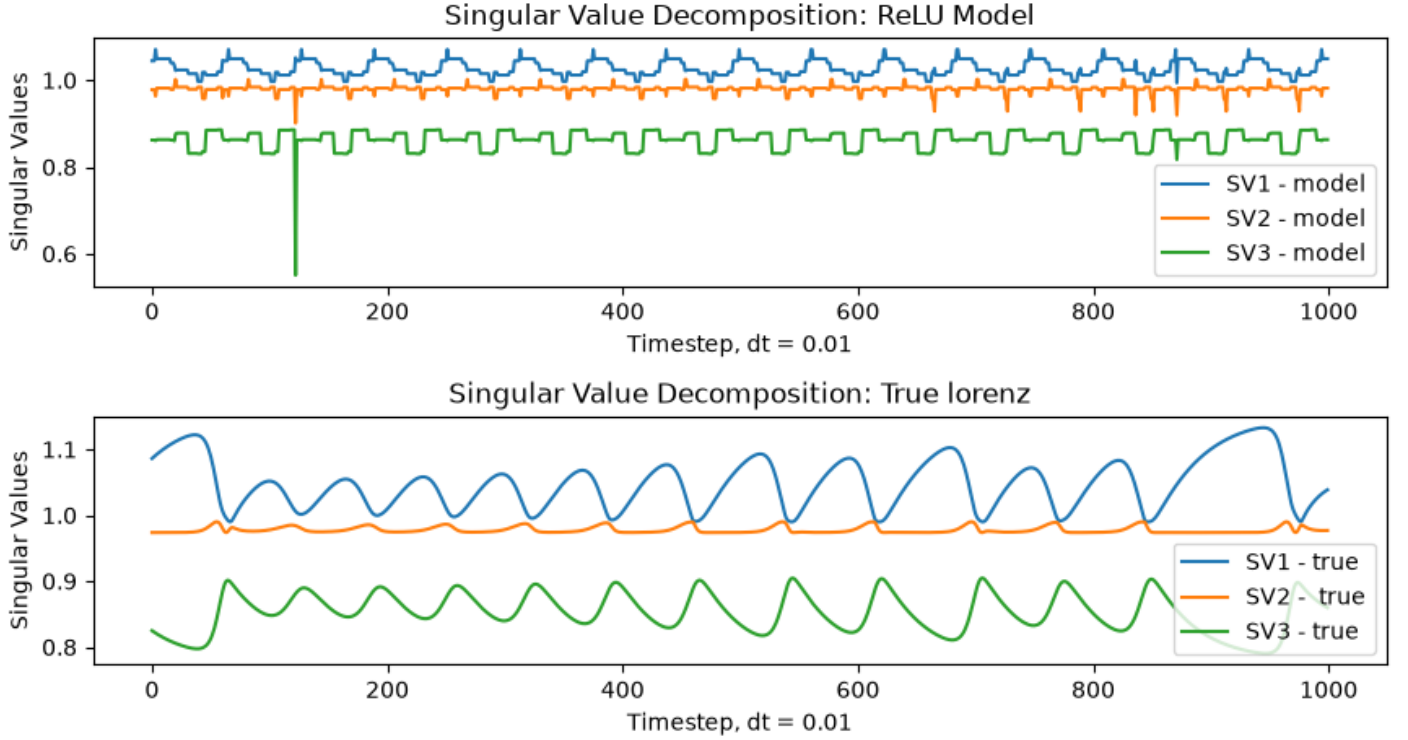


Figure 3: Singular value Decompositions for the ReLU Model (Top) and true Lorenz equations (Bottom)

3.2 Tanh

Next we will investigate tanh, the most common and assumed best activation function for Lorenz-63, shown in Figure 2.

Initially, we notice tanh is continuous and smooth, with a smooth derivative which peaks at 0 and asymptotically approaches 0 as $x \rightarrow \pm\infty$. This ensures we will have a continuous change in Jacobian unlike with ReLU.

Indeed, SVD and Lyapunov Spectra of tanh models proved more accurate than ReLU models. 100 random seeds for tanh models were trained to convergence, all with width 16. The results showed 99% of models had $\hat{\lambda}_1$, the first Lyapunov exponent, within 5% of the true Lyapunov exponent, $\lambda_1 = 0.906$, and 78% had $\hat{\lambda}_3$ within 5% of the true λ_3 , approximately -14.6. λ_3 often proved harder for models to achieve accurately. These results are promising.

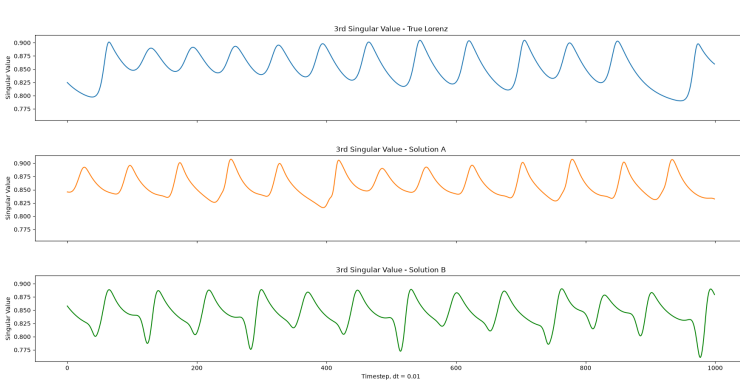
However, inspection of the SVD showed evidence that models could be separated into 2 main solutions. These 2 solutions are shown in Figure 4a. In one, the SVD showed smooth oscillations of amplitudes that increase with the radius of the loop, just as we'd hope. On the other hand, many models showed σ_3 decreasing slowly, then a sharp drop in fold areas. This shows under-contraction, until an overcorrection to allow for the change of lobe necessary at these points. This often results in $\hat{\lambda}_3 \nmid \lambda_3$. We can imperfectly estimate this behaviour by finding the minimum gradient of $\hat{\sigma}_3$. The true Lorenz system gives us a minimum σ_3 of roughly -0.262. The % of 16-width models with a minimum gradient of $\hat{\sigma}_3 < -0.4$, chosen through inspection, is 47%. This measure is not perfect but indicates the problem well. Interestingly, we see, in Figure 4b, that these solutions are not correlated to the MSE of the model. This implies accurate local geometry is not correlated to the difference between the prediction and the true trajectory.

This failure mode can be explained by the curl of the activation function. The derivative of the $\tanh(z)$ is $\text{sech}^2(z)$. We know each neuron contributes a rank-1 matrix to the Jacobian. At these fold points, certain neurons are specialised for the strong contraction. At these points, the absolute value of the neuron changes quickly, causing the derivative of the activation function to drop quickly. This is a result of the strong slope of $\text{sech}^2(z)$. When the derivative drops quickly, the effect on the Jacobian and $\hat{\sigma}_3$ is strong, emphasised by the low number of contributing neurons. This results in a sharp drop in $\hat{\sigma}_3$ referring to an area of high contraction. This drop is mindfully placed, to align with the areas for which strong contraction is needed for lobe-swapping.

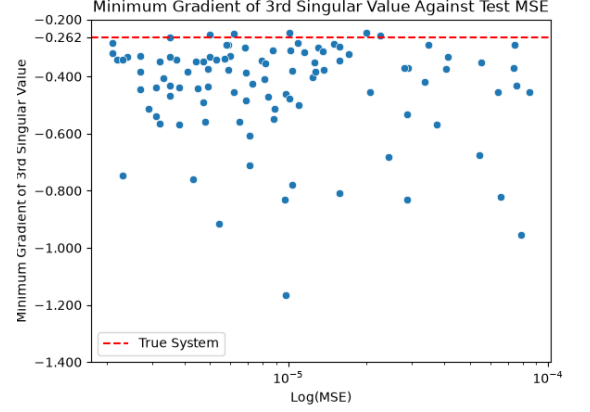
The other common basin systematically under-contracts the geometry, meaning the minima of $\hat{\sigma}_3$ is not as negative as the true system, however this does not seem to have an affect on the statistics of the attractor.

We will look into a particular model to prove this point. This model is chosen from the 100 random seeds to show the sharp drops in $\hat{\sigma}_3$. We identify, by ablation, that the 13th neuron is specialised for contraction in the direction we are interested in. Before the sharp drop, we keep the derivative of the neuron as constant, and plot both the fixed and true $\hat{\sigma}_3$, as well as the fixed and true $\text{sech}^2(z_{13})$, the derivative of the neuron. We can see that the $\text{sech}^2(z_{13})$ drops from 0.86 to 0.26 quickly. This is seen in Figure 5, and shows that without the quick drop in the derivative, the third singular value avoids the sharp drop.

To confirm this hypothesis, we investigate another activation function, softplus, for which the gradient of the derivative is less steep. We predict the SVD of this function would be smoother and more representative of the true system.



(a) Comparison of $\hat{\sigma}_3$ of the 2 categories of solution, with σ_3 , the true value. (Top) σ_3 , the true 3rd Singular Value (Middle) $\hat{\sigma}_3$ of a model showing smooth evolution, mirroring the true system. (Bottom) $\hat{\sigma}_3$ of a model showing sharp drops.



(b) Minimum Gradient of the 3rd Singular Value, $\hat{\sigma}_3$, of 100 trained models, with width 16 and a tanh activation.

Figure 4: Tanh activation analysis

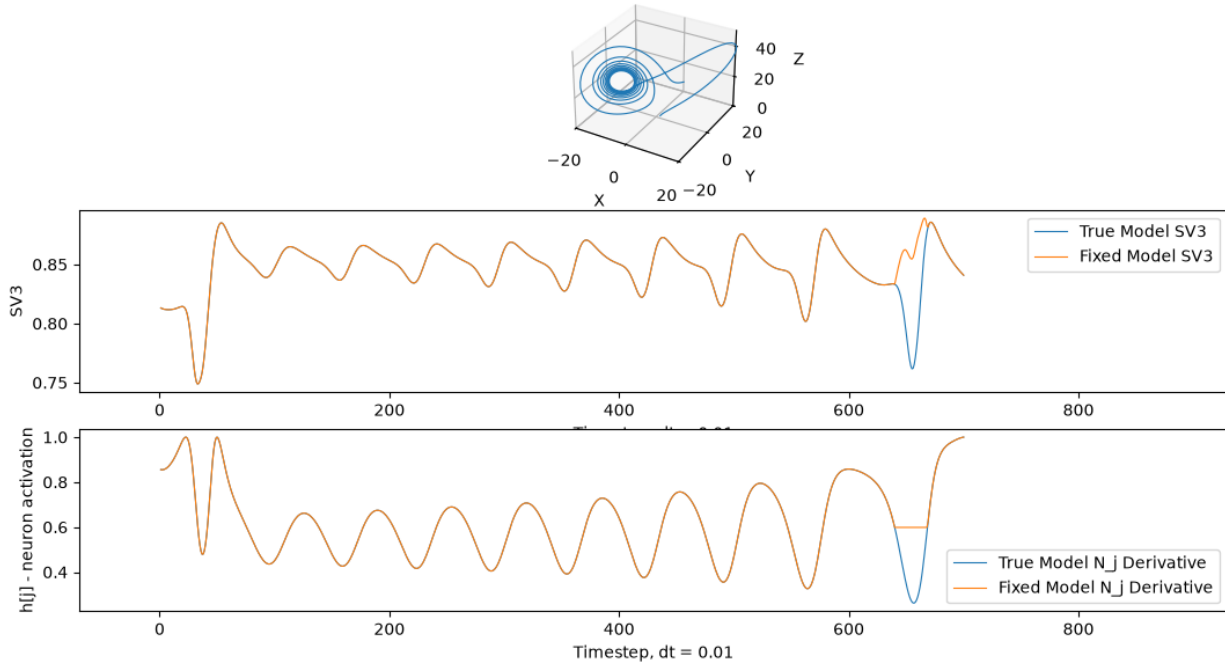


Figure 5: (Top) Autoregressive trajectory of the tanh model. (Middle) $\hat{\sigma}_3$ of the model, comparing fixed and true value. (Bottom) $\text{sech}^2(z_{13})$, derivative of a neuron specialised for contraction.

3.3 Softplus

We will show Softplus is a better activation function to consistently capture the local geometry of the Lorenz system. Softplus is defined as:

$$\text{Softplus}(z) = \frac{1}{\beta} \ln(1 + \exp(\beta z))$$

This makes the derivative:

$$\frac{d}{dz} \text{Softplus}(z) = \frac{1}{1 + \exp(-\beta z)}$$

For this study, we take $\beta = 1$.

Figure 6 compares the $\tanh(z)$ to $\text{softplus}(z)$, and $\frac{d}{dz} \tanh(z)$ to $\frac{d}{dz} \text{softplus}(z)$. We clearly see the gradient of $\frac{d}{dz} \text{softplus}(z)$, with an absolute maximum gradient of 0.25, is less steep than $\text{sech}^2(z)$, which has a maximum absolute gradient of roughly 0.77.

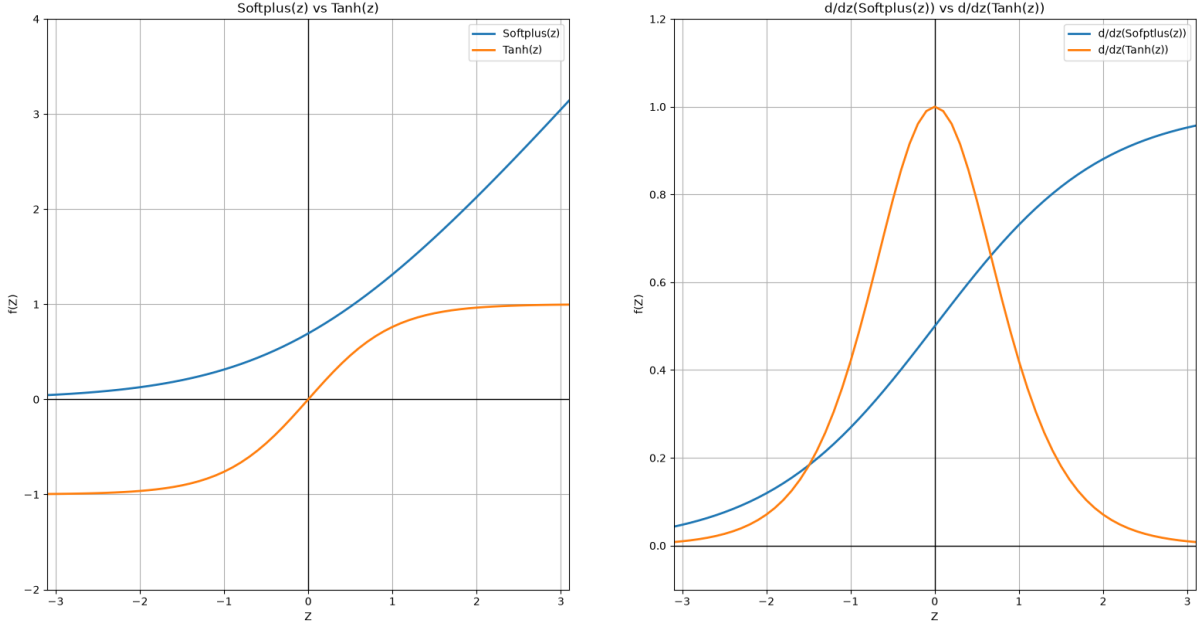
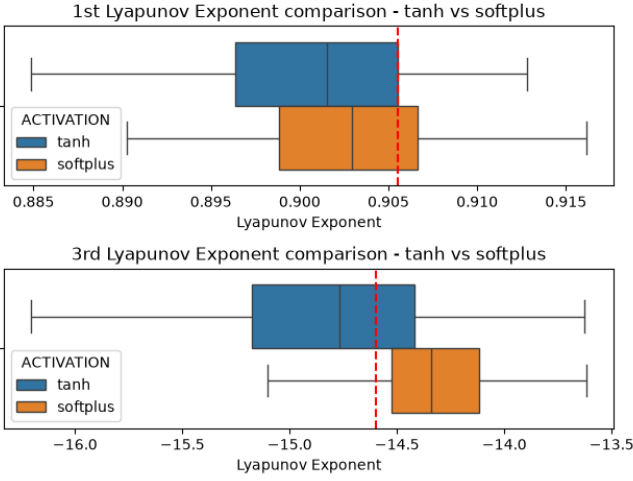
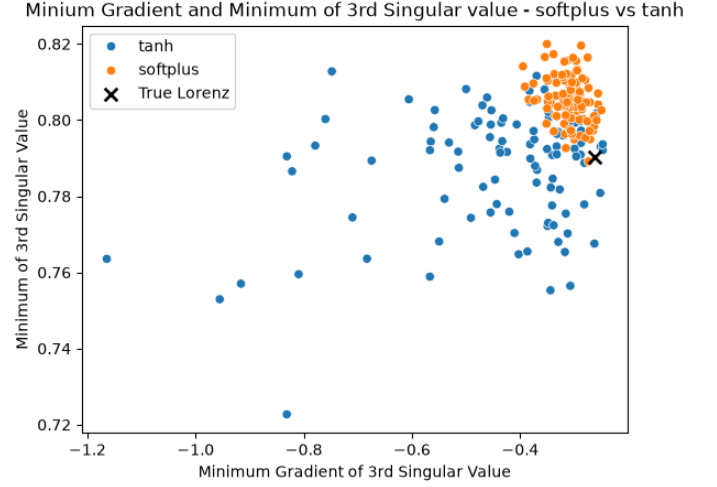


Figure 6: (Left) $\text{Softplus}(z)$ vs $\text{Tanh}(z)$. (Right) $\frac{d}{dz}(\text{Softplus}(z))$ vs $\frac{d}{dz}(\tanh(z))$

I similarly trained 100 models with the softplus activation. 100% of these models had $\hat{\sigma}_1$ within 5% of the true value, σ_1 . 91% of the model's had $\hat{\sigma}_3$ within 5% of the true value, σ_3 . Most importantly, 0% of the softplus models had a minimum gradient of less than -0.4, completely avoiding the failure mode of sharp corrections in contraction. Systematically, the softplus models would under-contrast, shown by a minimum $\hat{\sigma}_3$. Here, 99% of models had a minimum less negative than the true system. This can be seen in Figure 7. However, as mentioned earlier, this doesn't seem to have an affect on attractor statistics. It is understandable to have this shortcoming, as to have high contraction at key points, we need the derivative of the neurons to change quickly, exactly the problem we try to avoid. It seems this is an trade-off problem, which show evidence of the shortcomings of MLPs.



(a) (Top) $\hat{\sigma}_1$ distribution of tanh and softplus models. σ_1 represented by red dashed line. Outliers not shown. (Bottom) $\hat{\sigma}_3$ distribution of tanh and softplus models. σ_3 represented by red dashed line. Outliers not shown.



(b) Minimum gradients and minima of $\hat{\sigma}_3$ for softplus and tanh models, width

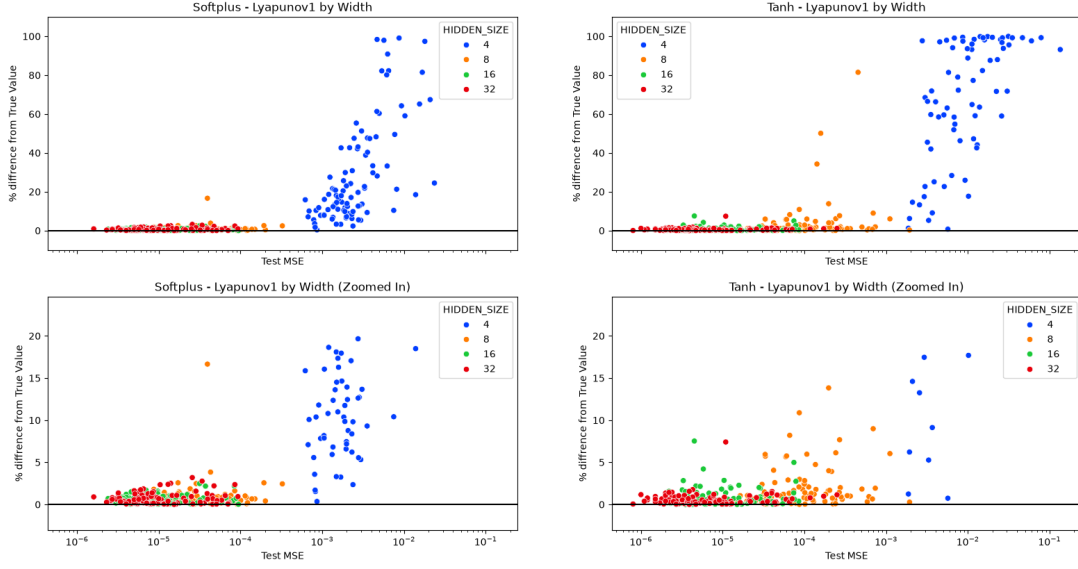
Figure 7: Tanh and Softplus Comparison

4 Width

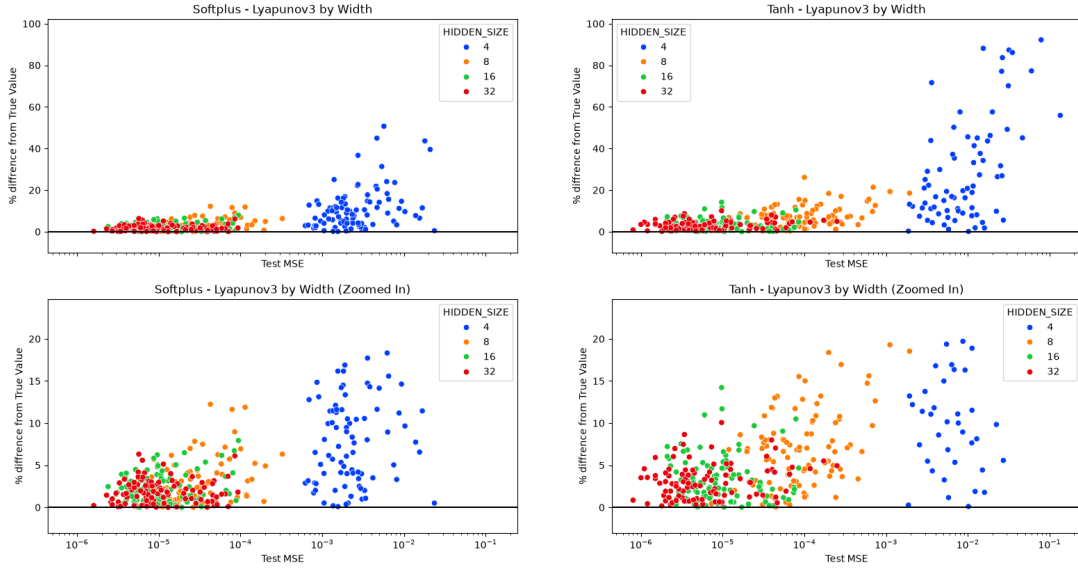
The width of an MLP refers to how many neurons are in its hidden layer. It is intuitive that the wider the model, the more likely it is to be able to reproduce the chaos and local geometry of the Lorenz equations. The results agree, with some interesting mechanistic findings. The table below shows the percentage of each width and each activation function to achieve metrics within a quantile bound of that of the true system. We look at them metrics of λ_1 , λ_3 , and the minimum gradient of σ_3 .

MODEL	$\lambda_1 - 5\%$	$\lambda_1 - 2\%$	$\lambda_3 - 5\%$	$\lambda_3 - 2\%$	Min Grad $\sigma_3 - 20\%$	Min Grad $\sigma_3 - 10\%$
Softplus, 4	7%	3%	33%	11%	3%	2%
Tanh, 4	2%	2%	10%	5%	1%	0%
Softplus, 8	99%	93%	83%	51%	61%	22%
Tanh, 8	85%	72%	35%	12%	33%	19%
Softplus, 16	100%	97%	91%	48%	62%	24%
Tanh, 16	99%	89%	78%	39%	19%	12%
Softplus, 32	100%	93%	96%	61%	79%	14%
Tanh, 32	99%	99%	85%	37%	27%	14%

Before plotting, we must remove extreme values, which did not converge to a chaotic system. This only occurs in width 4 models, in 1% of softplus models and 27% of tanh models. These are removed from future analysis.



(a) (Top Left) Percentage error of $\hat{\lambda}_1$ from the true value, for softplus models across widths. (Bottom Left) Percentage error of $\hat{\lambda}_1$ from the true value, for softplus models across widths. (zoomed in at $< 20\%$). (Top Right) Percentage error of $\hat{\lambda}_1$ from the true value, for tanh models across widths. (Bottom Right) Percentage error of $\hat{\lambda}_1$ from the true value, for tanh models across widths. (zoomed in at $< 20\%$).



(b) (Top Left) Percentage error of $\hat{\lambda}_3$ from the true value, for softplus models across widths. (Bottom Left) Percentage error of $\hat{\lambda}_3$ from the true value, for softplus models across widths. (zoomed in at $< 20\%$). (Top Right) Percentage error of $\hat{\lambda}_3$ from the true value, for tanh models across widths. (Bottom Right) Percentage error of $\hat{\lambda}_3$ from the true value, for tanh models across widths. (zoomed in at $< 20\%$).

Figure 8: Lyapunov Spectra Analysis on Widths

By inspecting Figure 8, we can clearly see that in both Lyapunov Exponents, width-4 models have considerably greater variation than wider models, and are less likely to preserve the correct long-term dynamics. In tanh models, there seems a clearer dependence on width: in softplus models, particularly for $\hat{\lambda}_1$, models of widths 8, 16 and 32 are of comparable consistency. In tanh models, we see that consistency increases with width. However, in all cases, the best performing models of each width are of comparable accuracy. It is therefore the consistency of finding a good solution that is influenced by width, rather than the quality of best solution.

There are important differences between $\hat{\lambda}_1$ and $\hat{\lambda}_3$. We can see the latter is more variable across seeds, suggesting it is more difficult to reproduce. In the tanh models, we find a clear trend of increasing width improving the likelihood of capturing $\hat{\lambda}_3$. In softplus models, however, apart from the clear separation of width-4 and the higher widths, the increase in consistency is comparatively small.

These plots again back up the conclusion that softplus is more consistently able to capture the dynamics of the Lorenz equations than tanh.

We next investigate the common solutions capturing in the "Activation" section, namely sharp drops in $\hat{\sigma}_3$ represented by a small minimum gradient of $\hat{\sigma}_3$, and a systematic under-contraction represented by a minimum $\hat{\sigma}_3$ higher than the true system. These plots remove extreme values, namely where the minimum of $\hat{\sigma}_3 \leq 0.5$. These values were only provided by width-4 models. This can be seen in Figure 9.

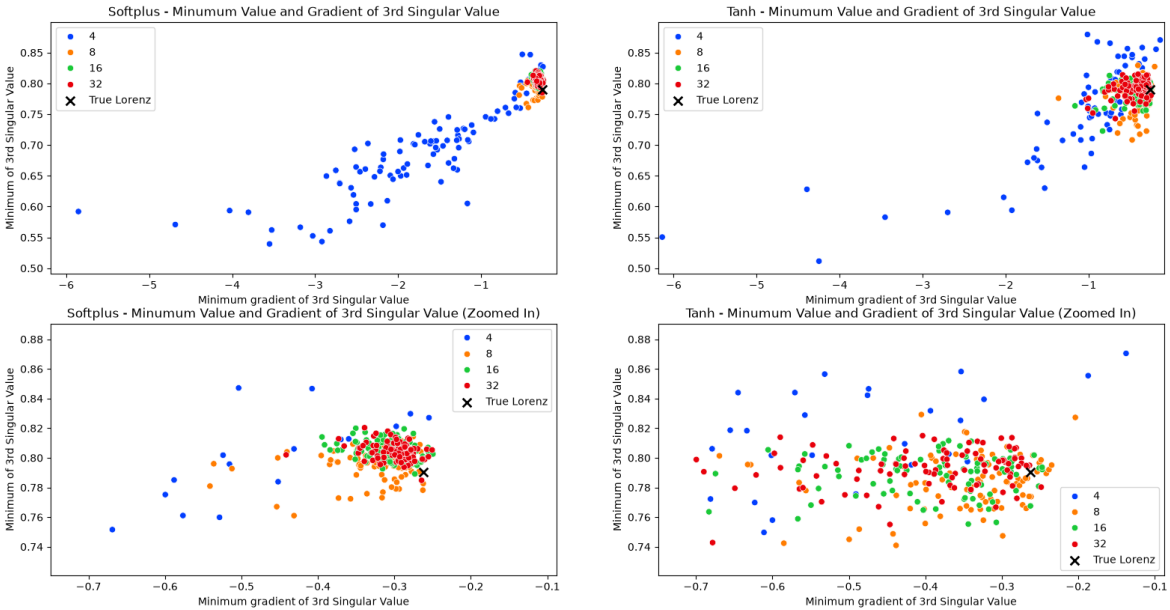


Figure 9: (Top Left) Minimum gradient and minimum of $\hat{\lambda}_3$ for softplus models across widths. (Bottom Left) Minimum gradient and minimum of $\hat{\lambda}_3$ for softplus models across widths (zoomed in). (Top Right) Minimum gradient and minimum of $\hat{\lambda}_3$ for tanh models across widths. (Bottom Right) Minimum gradient and minimum of $\hat{\lambda}_3$ for softplus models across widths (zoomed in).

We, as usual, notice a substantial difference between width-4 models and the larger models, with width 4 having much more variability and, in general, much less accurate values. Unlike the Lyapunov Spectra, the best performing models are noticeably worse than almost all larger models. We notice that for width-4 models, even in softplus, the values almost all have a minimum gradient of $\hat{\sigma}_3$ less than the true value. This is the same failure mode as diagnosed earlier with the tanh model. As before this can be explained by the contributions to the Jacobian. Whereas before we had a sharp drop in a particular neuron, which was specialised for contraction, this sharp drop can be explained by the overall drop in the derivative of neurons. With such a low number of neurons, each rank-1 contribution is more influential on the overall Jacobian. At fold points, even a small drop in the derivative of neurons provides a big impact on the Jacobian, resulting in a sharp change in the singular values, and in particular a sharp drop in $\hat{\sigma}_3$.

In the zoomed in plots, we can see that the softplus models has much less variability than the tanh models, and in general are closer to the true values. Indeed, as seen earlier, for each width, the tanh models have a lot more models in which the minimum gradient of $\hat{\sigma}_3$ is much lower than the true system. That being said, the best performing model of each are of similar accuracy.

We notice, in the softplus model, the minimum of $\hat{\sigma}_3$ is almost always higher than the true system. This was touched on earlier, and represents a systematic under-contraction of the model, however it is not clear that it has any effect on the statistics of the attractor. This shortcoming can be attributed to the trade-off between contraction capability and alleviation of sharp drops in $\hat{\sigma}_3$, which is an inherent problem of small MLP's in this case. It can be seen that the best performing model is of width 8, despite, in general, being worse. It is theorised that this is because the increased influence on each contributed no neuron to

the Jacobian actually helps the model in this case, allowing a stronger drop than is allowed by larger models using softplus.

The minimum gradient of tanh models does not seem to improve with width. In fact, from Table (x), 33% of the width-8 models are within 10% error of the true minimum σ_3 , as oppose to 19% and 27% of width-16 and width-32 models respectively. Unlike softplus, it can be seen that the tanh models are roughly split between below and above the minimum gradient of σ_3 . As mentioned earlier, these refer to the 2 possible categories that a model tends to show in the SVD decomposition.

APPENDIX

Derivation of the Tangent Propagator

We have an unperturbed point, $\bar{x}$, and perturb it with the perturbation x' , so the perturbed point, x is:

$$x(t) = \bar{x}(t) + x'(t)$$

First, for sufficiently small perturbations, dynamics may be linearised. Thus, we have $\dot{x}'(t) = J(t)x'(t)$. Here, $J(t) = F'(x)|_{x=x(t)}$. The tangent propagator, $M(t, t_0)$, is the map which takes a the perturbation, x' , at time t_0 to the perturbation at time t :

$$x'(t) = M(t, t_0)x'(t_0)$$

As choice of initial time is arbitrary, we set $t_0 = 0$. We have:

$$x'(t) = M(t, 0)x'(0)$$

$$\dot{x}' = \frac{dM}{dt}x'(0)$$

$$J(t)x'(t) = \frac{dM}{dt}x'(0)$$

$$\frac{dM}{dt} = J(t)M$$

Then, by integrating:

$$\int_0^t \frac{dM(s, 0)}{ds} ds = \int_0^t J(s)M(s, 0) ds$$

In general, the multiplication of Jacobians does not commute:

$$J(t_1)J(t_2) \neq J(t_2)J(t_1)$$

Hence, we must preserve the order of multiplication.

$$M(t, 0) - M(0, 0) = \int_0^t J(s)M(s, 0) ds$$

. Notice $M(0, 0) = I$. We substitute:

$$M(t, 0) = I + \int_0^t J(s_1) \left[I + \int_0^{s_1} J(s_2)M(s_2, 0) ds_2 \right] ds_1$$

$$M(t, 0) = I + \int_0^t J(s_1) ds_1 + \int_0^t \int_0^{s_1} J(s_1)J(s_2)M(s_2, 0) ds_2 ds_1$$

Repeating this substitution, we obtain the Dyson Series, which can be written compactly:

$$M(t, 0) = \mathcal{T} \exp \left(\int_0^t J(s) ds \right),$$

where $\mathcal{T}$ is the time-ordering operator. Recall, the first Lyapunov Exponent, λ_1 , is the long-term averaged exponential rate of growth of a perturbation:

$$\|x'(t)\| \sim e^{\lambda_1 t} \|x'(0)\|$$

Rearranging, and finding the limit:

$$\lambda_1 = \lim_{t \rightarrow \infty} \frac{1}{t} \ln \frac{\|x'(t)\|}{\|x'(0)\|}$$

Hence, we have

$$\lambda_1 = \lim_{t \rightarrow \infty} \frac{1}{t} \ln \frac{\|M(t, 0)x'(0)\|}{\|x'(0)\|}$$